🧪 GUÍA DE LABORATORIO DE SIMULACIÓN CAPITULO 6 - Corke

Capítulo 6: Localización y Mapeo

1. Introducción

En esta guía se estudiarán los conceptos fundamentales de localización y mapeo en robótica móvil utilizando Python, Robotics Toolbox y ROS2.

Durante la práctica se implementarán:

- Odometría
- Estimación usando EKF
- Corrección con landmarks
- SLAM con ROS2
- Visualización en RViz2

---

2. Objetivos de Aprendizaje

- Explicar y modelar la localización por odometría y sus limitaciones
- Implementar EKF para estimación de pose
- Integrar observaciones de landmarks para corrección
- Comprender SLAM y su complejidad
- Implementar pipelines reales en ROS2 (EKF, AMCL, SLAM)

---

3. Resultados Esperados (Docente)

Al finalizar, el estudiante debe:

- Mostrar trayectorias real vs estimada
- Evidenciar crecimiento de incertidumbre sin sensores
- Demostrar reducción de incertidumbre con landmarks
- Ejecutar localización/SLAM en ROS2 con visualización en RViz2

---

4. Parte 1: Odometría (Dead Reckoning)

Objetivo

Implementar una simulación básica de odometría y analizar el crecimiento del error.

Código base

```
from roboticstoolbox import Bicycle
import numpy as np
```

```
import matplotlib.pyplot as plt

# Covarianza del ruido: incertidumbre para estimación de pose
V = np.diag([0.2, np.deg2rad(0.5)])**2
print(V)

# Robot con ruido
robot = Bicycle(covar=V)

# Robot ideal (sin ruido)
robot_ideal = Bicycle(covar=np.zeros((2,2)))

# Variables para almacenar trayectorias
x_real = []
y_real = []

x_ideal = []
y_ideal = []
qp_real=[0, 0, 0]
qp_ideal=[0, 0, 0]

# Simulación
for _ in range(200):

    # Movimiento robot real
    odo_real=robot.step((1.0, 0.2))
    qp_real=robot.f(qp_real,odo_real)

    # Movimiento robot ideal
    odo_ideal= robot_ideal.step((1.0, 0.2))
    qp_ideal=robot.f(qp_ideal,odo_ideal)

    # Guardar trayectoria real
    x_real.append(qp_real[0])
    y_real.append(qp_real[1])

    # Guardar trayectoria ideal
    x_ideal.append(qp_ideal[0])
    y_ideal.append(qp_ideal[1])

# Graficar trayectorias
plt.figure(figsize=(8,6))

plt.plot(
    x_ideal,
    y_ideal,
    label='Trayectoria estimada con odometría sin ruido',
    linewidth=1
```

```
)

plt.plot(
   x_real,
   y_real,
   label='Trayectoria estimada con odometría con ruido',
   linewidth=1
)

plt.xlabel('X [m]')
plt.ylabel('Y [m]')
plt.title('Comparación de trayectorias')
plt.grid(True)
plt.axis('equal')
plt.legend()

plt.show()
```

Gráfica esperada

- Trayectoria suave pero con desviación acumulada
- Error creciente con el tiempo

Indicadores de logro

- Identifica crecimiento del error
- Relaciona error con ruido

Preguntas de análisis

1. ¿Por qué la trayectoria presenta error?
2. ¿Qué representa la matriz de covarianza V?
3. ¿Qué ocurre si aumenta el ruido?

---

5. Parte 2. Estimación usando EKF (Extended Kalman Filter)

Objetivo

Implementar estimación probabilística de pose usando un filtro de Kalman extendido.

Código:

```
from roboticstoolbox import Bicycle, EKF, RandomPath
import numpy as np
import matplotlib.pyplot as plt
```

```
# Covarianza odometría
V = np.diag([0.02, np.deg2rad(0.5)])**2

# Robot
robot = Bicycle(covar=V, animation=None)

# Trayectoria "aleatoria"
robot.control = RandomPath(workspace=10)

# Covarianza inicial
P0 = np.diag([0.05, 0.05, np.deg2rad(0.5)])**2

# EKF
ekf = EKF(robot=(robot, V), P0=P0)

# Ejecutar simulación
ekf.run(T=20)

# Trayectoria
plt.figure(figsize=(8,6))
robot.plot_xy(color='blue')
ekf.plot_xy(color='red')
plt.title('Trayectoria real vs estimada')
plt.grid(True)

# Incertidumbre
plt.figure(figsize=(8,6))
plt.plot(ekf.get_t(), ekf.get_Pnorm(), linewidth=2)
plt.xlabel('Tiempo [s]')
plt.ylabel('Incertidumbre')
plt.title('Crecimiento de incertidumbre')
plt.grid(True)

plt.show()
```

Resultado esperado

Los estudiantes deben observar:

- Divergencia entre trayectoria real y estimada

- Crecimiento continuo de incertidumbre

- Ausencia de corrección de pose

Preguntas de análisis

1. ¿Por qué el EKF no logra corregir el error?

2. ¿Qué representa la curva de incertidumbre?

3. ¿Qué ocurre si aumenta el valor de V?

---

6. Parte 3 – EKF con Landmarks

Objetivo

Corregir la pose usando observaciones externas del entorno.

Código:

```
from roboticstoolbox import *
import numpy as np
import matplotlib.pyplot as plt
from math import pi

# Covarianza odometría
V = np.diag([0.02, np.deg2rad(0.5)])**2

# Robot
robot = Bicycle(covar=V, animation=None)
robot.control = RandomPath(workspace=10)

# Mapa de landmarks
mapa = LandmarkMap(20, workspace=10)

# Covarianza sensor de rango
W = np.diag([0.1, np.deg2rad(1)])**2

# Sensor de rango, tipo Lidar
sensor = RangeBearingSensor(
   robot=robot,
   map=mapa,
   covar=W,
   angle=[-pi/2, pi/2],
   range=4
)
```

```
# Covarianza inicial
P0 = np.diag([0.05, 0.05, np.deg2rad(0.5)])**2

# EKF
ekf = EKF(
   robot=(robot, V),
   P0=P0,
   map=mapa,                              #Conoce el mapa  de landmarks previamente
   sensor=(sensor, W)
)

# Simulación
ekf.run(T=40)

# Trayectorias
plt.figure(figsize=(8,6))
mapa.plot()
robot.plot_xy(color='blue')
ekf.plot_xy(color='red')
ekf.plot_ellipse()
plt.title('EKF con landmarks')
plt.grid(True)

# Incertidumbre
plt.figure(figsize=(8,6))
plt.plot(ekf.get_t(), ekf.get_Pnorm(), linewidth=2)
plt.xlabel('Tiempo [s]')
plt.ylabel('Incertidumbre')
plt.title('Incertidumbre con correcciones')
plt.grid(True)

plt.show()
```

Resultado esperado

Los estudiantes deben observar:

- Mejor ajuste entre trayectoria real y estimada

- Reducción periódica de incertidumbre

- Elipses de confianza más pequeñas

Preguntas de análisis

1. ¿Qué es la innovación?

2. ¿Por qué disminuye la incertidumbre?

3. ¿Qué ocurre cuando no se observan landmarks?

---

7. Parte 4 – Introducción a SLAM (Simultaneus Localization and mapping)

Objetivo

Comprender el problema de localización y mapeo simultáneo (SLAM) mediante una simulación en Python.

Código:

```
from roboticstoolbox import *
import numpy as np
import matplotlib.pyplot as plt
from math import pi

# ==============================
# Covarianza de odometría
# ==============================

V = np.diag([
   0.02,
   np.deg2rad(0.5)
])**2

# ==============================
# Robot
# ==============================

robot = Bicycle(
   covar=V,
   animation=None
)

robot.control = RandomPath(workspace=10)

# ==============================
# Mapa real
# ==============================
```

```
mapa = LandmarkMap(20, workspace=10)

# ==============================
# Sensor
# ==============================

W = np.diag([
   0.1,
   np.deg2rad(1)
])**2

sensor = RangeBearingSensor(
   robot=robot,
   map=mapa,
   covar=W,
   angle=[-pi/2, pi/2],
   range=4
)

# ==============================
# Covarianza inicial
# ==============================

P0 = np.diag([
   0.05,
   0.05,
   np.deg2rad(0.5)
])**2

# ==============================
# EKF SLAM
# ==============================

slam = EKF(
   robot=(robot, V),
   P0=P0,
   sensor=(sensor, W),
   map=None                              #No conoce el mapa  de landmarks previamente
)

# ==============================
# Ejecutar simulación
# ==============================
```

```
slam.run(T=40)

# ==============================
# Graficar resultados
# ==============================

plt.figure(figsize=(10,8))

# Mapa real
mapa.plot()

# Trayectoria real
robot.plot_xy(
   color='blue',
   label='Trayectoria real'
)

# Trayectoria estimada
slam.plot_xy(
   color='red',
   label='Trayectoria estimada'
)

# Elipses de confianza
slam.plot_ellipse()

plt.title('SLAM usando EKF')
plt.grid(True)
plt.axis('equal')

# ==============================
# Incertidumbre
# ==============================

plt.figure(figsize=(8,6))

plt.plot(
   slam.get_t(),
   slam.get_Pnorm(),
   linewidth=2
)

plt.xlabel('Tiempo [s]')
```

```
plt.ylabel('Incertidumbre')
plt.title('Evolución de incertidumbre en SLAM')
plt.grid(True)

plt.show()
```

Resultado esperado
Los estudiantes deben observar:
- Construcción progresiva del mapa
- Corrección de trayectoria
- Estimación simultánea de pose y landmarks
- Reducción parcial de incertidumbre
- Diferencia entre trayectoria real y estimada
- Comparación con el resultado de la anterior simulación (EKF

Conceptos importantes

¿Qué está resolviendo SLAM?

El robot debe estimar simultáneamente:
- su pose
- el mapa
sin conocer previamente ninguno de los dos.

Preguntas de análisis

1. ¿Por qué SLAM es más complejo que localización?
2. ¿Qué ocurre si aumenta el ruido del sensor?
3. ¿Por qué la incertidumbre no desaparece completamente?
4. ¿Qué ventajas ofrece SLAM frente a usar un mapa conocido?
5. ¿Qué aplicaciones industriales podrían usar SLAM?
6. ¿Cómo es la comparación de los resultados obtenidos, conociendo y sin conocer el mapa de landmarks previamente?

---

8. Parte 5 – SLAM con Cartographer usando ROS2 Jazzy + Turtlebot3 + Gazebo + RViz2
Este procedimiento está ajustado específicamente para:
- Ubuntu 24.04
- ROS2 Jazzy Jalisco

- Gazebo Harmonic
- Turtlebot3
- Cartographer ROS2
- Nav2

Si ya tienen instalado todo, puede pasar al paso 8.7.

🔷 8.1. Verificar ROS2 Jazzy

Ver versión
ros2 --version
Debe aparecer algo similar a:
ros2 0.32.x

Ver distribución
echo $ROS_DISTRO
Resultado esperado:
jazzy

🔷 8.2. Instalar dependencias

Actualizar paquetes
sudo apt update

🔷 8.3. Instalar Turtlebot3

---

sudo apt install ros-jazzy-turtlebot3*

🔷 8.4. Instalar Gazebo Harmonic

En ROS2 Jazzy ya NO se usa Gazebo Classic.
Se usa:
Gazebo Harmonic

Instalar simulador
sudo apt install ros-jazzy-ros-gz

Instalar bridge ROS ↔ Gazebo
sudo apt install ros-jazzy-ros-gz-bridge

🔷 8.5. Instalar Cartographer

---

sudo apt install ros-jazzy-cartographer

---

sudo apt install ros-jazzy-cartographer-ros

🔷 8.6. Instalar Navigation2

```
sudo apt install ros-jazzy-navigation2
```

```
sudo apt install ros-jazzy-nav2-bringup
```

🔷 8.7. Crear workspace

```
mkdir -p ~/ros2_ws/src
```

```
cd ~/ros2_ws
```

🔷 8.8. Crear paquete

```
cd src
```

```
ros2 pkg create localization_lab --build-type ament_python
```

🔷 8.9. Estructura final

```
localization_lab/
 ├── package.xml
 ├── setup.py
 ├── setup.cfg
 ├── resource/
 |   └── localization_lab
 ├── localization_lab/
 |    ├── __init__.py
 |   └── simple_controller.py
 ├── launch/
 |   └── cartographer.launch.py
 ├── config/
 |   └── cartographer.lua
 └── rviz/
     └── slam.rviz
```

🔷 8.10. Nodo controlador

Puede crear el siguiente simple_controller.py o **usar el controlador reactivo creado en la práctica de ROS2 del capítulo 5**.

Crear archivo

```
cd localization_lab/localization_lab
```

---

touch simple_controller.py

Código completo

```
import rclpy
from rclpy.node import Node

from geometry_msgs.msg import TwistStamped

class SimpleController(Node):

    def __init__(self):

        super().__init__('simple_controller')

        self.publisher_ = self.create_publisher(
            TwistStamped,
            '/cmd_vel',
            10
        )

        self.timer = self.create_timer(
            0.1,
            self.move_robot
        )

    def move_robot(self):

        msg = TwistStamped()

        msg.header.stamp = self.get_clock().now().to_msg()
        msg.header.frame_id = 'base_link'

        msg.twist.linear.x = 0.2
        msg.twist.angular.z = 0.1

        self.publisher_.publish(msg)

def main(args=None):

    rclpy.init(args=args)

    node = SimpleController()

    rclpy.spin(node)

    node.destroy_node()
```

```
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

🔷 11. setup.py para Jazzy

Editar
gedit ~/ros2_ws/src/localization_lab/setup.py

Código completo

```
from setuptools import setup
from glob import glob
import os

package_name = 'localization_lab'

setup(
name=package_name,

version='0.0.1',

packages=[package_name],

data_files=[

(
'share/ament_index/resource_index/packages',
['resource/' + package_name]
),

(
'share/' + package_name,
['package.xml']
),

(
os.path.join('share', package_name, 'launch'),
glob('launch/*.launch.py')
),

(
os.path.join('share', package_name, 'config'),
glob('config/*.lua')
),
```

```
(
os.path.join('share', package_name, 'rviz'),
glob('rviz/*.rviz')
),
],

install_requires=['setuptools'],

zip_safe=True,

maintainer='docente',

maintainer_email='docente@lab.com',

description='Localization and SLAM laboratory',

license='Apache-2.0',

tests_require=['pytest'],

entry_points={
'console_scripts': [
],
},
)
```

🔷 12. package.xml

Editar

```
gedit ~/ros2_ws/src/localization_lab/package.xml
```

Añadir dependencias

```
<depend>rclpy</depend>
<depend>geometry_msgs</depend>
<depend>sensor_msgs</depend>
<depend>nav_msgs</depend>
<depend>cartographer_ros</depend>
```

🔷 13. Crear configuración Cartographer

Crear carpeta config

```
mkdir ~/ros2_ws/src/localization_lab/config
```

IMPORTANTE

Antes debes copiar:

cp /opt/ros/jazzy/share/cartographer/configuration_files/map_builder.lua ~/ros2_ws/src/localization_lab/config/

cp /opt/ros/jazzy/share/cartographer/configuration_files/trajectory_builder.lua ~/ros2_ws/src/localization_lab/config/

cp /opt/ros/jazzy/share/cartographer/configuration_files/pose_graph.lua ~/ros2_ws/src/localization_lab/config/

Crear archivo
touch ~/ros2_ws/src/localization_lab/config/cartographer.lua

Código completo

```
include "map_builder.lua"
include "trajectory_builder.lua"

options = {

map_builder = MAP_BUILDER,

trajectory_builder = TRAJECTORY_BUILDER,

map_frame = "map",

tracking_frame = "base_scan",

published_frame = "base_footprint",

odom_frame = "odom",

provide_odom_frame = true,

publish_frame_projected_to_2d = true,

use_odometry = false,

use_nav_sat = false,

use_landmarks = false,

num_laser_scans = 1,

num_multi_echo_laser_scans = 0,

num_subdivisions_per_laser_scan = 1,

num_point_clouds = 0,
```

lookup_transform_timeout_sec = 0.2,

submap_publish_period_sec = 0.3,

pose_publish_period_sec = 0.005,

trajectory_publish_period_sec = 0.03,

rangefinder_sampling_ratio = 1.0,

odometry_sampling_ratio = 1.0,

fixed_frame_pose_sampling_ratio = 1.0,

imu_sampling_ratio = 1.0,

landmarks_sampling_ratio = 1.0,
}

MAP_BUILDER.use_trajectory_builder_2d = true

TRAJECTORY_BUILDER_2D.use_imu_data = false

TRAJECTORY_BUILDER_2D.min_range = 0.12

TRAJECTORY_BUILDER_2D.max_range = 3.5

TRAJECTORY_BUILDER_2D.missing_data_ray_length = 5.0

POSE_GRAPH.optimize_every_n_nodes = 35

return options

🔷 14. Crear launch file

Crear carpeta launch
mkdir ~/ros2_ws/src/localization_lab/launch

Crear archivo
touch ~/ros2_ws/src/localization_lab/launch/cartographer.launch.py

---

Código completo
from launch import LaunchDescription

from launch_ros.actions import Node

```
from ament_index_python.packages import get_package_share_directory

import os


def generate_launch_description():

    config_dir = os.path.join(
        get_package_share_directory('localization_lab'),
        'config'
    )

    cartographer_node = Node(

        package='cartographer_ros',

        executable='cartographer_node',

        output='screen',

        parameters=[
            {'use_sim_time': True}
        ],

        arguments=[
            '-configuration_directory',
            config_dir,

            '-configuration_basename',
            'cartographer.lua'
        ]
    )

    occupancy_grid_node = Node(

        package='cartographer_ros',

        executable='cartographer_occupancy_grid_node',

        output='screen',

        parameters=[
            {'use_sim_time': True}
        ],

        arguments=[
            '-resolution',
            '0.05'
```

```
    ]
  )

  return LaunchDescription([

    cartographer_node,

    occupancy_grid_node

  ])
```

🔷 15. Compilar workspace

Ir al workspace

```
cd ~/ros2_ws
```

Limpiar compilación vieja

```
rm -rf build install log
```

Compilar

```
colcon build
```

🔷 16. Cargar entorno

```
source install/setup.bash
```

🔷 17. Configurar Turtlebot3

```
export TURTLEBOT3_MODEL=burger
```

🔷 18. Lanzar Gazebo Harmonic

Terminal 1

```
ros2 launch turtlebot3_gazebo empty_world.launch.py
```

🔷 19. Verificar tópicos

Terminal 2

```
source ~/ros2_ws/install/setup.bash
```

```
ros2 topic list
```

Verificar

/scan
/odom

/tf
/cmd_vel

🔷 20. Ejecutar controlador

Terminal 3
source ~/ros2_ws/install/setup.bash

---

ros2 run localization_lab simple_controller

🔷 21. Ejecutar Cartographer

Terminal 4
source ~/ros2_ws/install/setup.bash

---

ros2 launch localization_lab cartographer.launch.py

🔷 22. Ejecutar RViz2

Terminal 5
rviz2

🔷 23. Configurar RViz2

Fixed Frame
map

Agregar Displays
Add → By Topic
Agregar:

- /map
- /scan
- /tf
- /submap_list

🔷 24. Resultado esperado

En Gazebo

- robot moviéndose

En RViz2

- nube LiDAR
- mapa construyéndose
- trayectoria

- submapas
- transformaciones TF

🔷 25. Guardar mapa

Instalar map saver
sudo apt install ros-jazzy-nav2-map-server

Guardar
ros2 run nav2_map_server map_saver_cli -f my_map

🔷 26. Concepto central

El sistema implementa el pipeline real usado por Turtlebot3:
LiDAR
→ Cartographer
→ SLAM
→ Mapa
→ RViz2